

“Electricity Bill Payment”



A

PBL Report

Submitted to the

SAGE University, Bhopal, M.P.

in partial fulfillment of the requirements for the award of the Degree of

BTech CSE

I Semester

By

Piyush Sahu – 230725

Tanishka Soni – 231065

Jatin Singh chouhan -231628

Vivek Chouhan-231528

Agniva Jana Bhattacharya - 232528

Under the Guidance of

Kapil Agrawal

PBL Guide

SCHOOL OF ADVANCE COMPUTING

SANJEEV AGRAWAL GLOBAL EDUCATIONAL UNIVERSITY, BHOPAL

AUTUMN 2023-24



**SANJEEV AGRAWAL GLOBAL EDUCATIONAL UNIVERSITY,
BHOPAL**

SCHOOL OF ADVANCE COMPUTING

CERTIFICATE

This is to certify that the work embodies in this project entitled “**Electricity Bill Payment**” being submitted by Pixyish Sahu-230725 Tanishka Soni-231065 Jatin Singh-231628 Vivek Singh Chouhan-231629 Agniva Jana Bhattacharya-232528 in partial fulfillment of the requirement for the award of the degree of **BTech CSE(FSD) to** School of ADVANCE COMPUTING, Sanjeev Agrawal Global Educational University, Bhopal (M.P) during the academic year **2023-24** is a record of bonafide piece of work, undertaken by him under the supervision of the undersigned.

Kapil Agrawal
PBL Teacher
(GUIDE)

Dr Pinaki Ghosh
HOD, SSOAC



SANJEEV AGRAWAL GLOBAL EDUCATIONAL UNIVERSITY, BHOPAL

SCHOOL OF ADVANCE COMPUTING

CERTIFICATE OF APPROVAL

The Project entitled “**Electricity Bill Payment**” being submitted by Piyush Sahu-230725 Tanishka Soni-231065 Jatin Singh-231628 Vivek Singh Chouhan-231629 Agniva Jana Bhattacharya-232528 has been examined by us and is hereby approved for the award of the degree of **BTech CSE(FSD)**, for which it has been submitted. It is understood that by this approval the undersigned do not necessarily endorse or approve any statement made, opinion expressed or conclusion drawn there in, but approve the project only for the purpose for which it has been submitted.

(Internal Examiner)

(External Examiner)



SANJEEV AGRAWAL GLOBAL EDUCATIONAL UNIVERSITY, BHOPAL

SCHOOL OF ADVANCE COMPUTING

DECLARATION

I hereby declare that the work, which is being presented in this project entitled **“Electricity Bill Payment”** for fulfillment of the requirements for the award of the degree of **BTech CSE(FSD)** submitted in the School of Advanced Computing, Sanjeev Agrawal Global Educational University, Bhopal, M.P. is an authentic record of my own work carried under the guidance of **“Kapil Agrawal”** I have not submitted the matter embodied in this report for the award of any other degree.

I also declare that “A check for Plagiarism has been carried out on this report and is found within the acceptable limit.”

Piyush Sahu – 230725
Tanishka Soni – 231065
Agniva Jana Bhattacharya - 232528
Jatin Singh Chouhan -231628
Vivek Singh Chouhan-231528



SANJEEV AGRAWAL GLOBAL EDUCATIONAL UNIVERSITY, BHOPAL

SCHOOL OF ADVANCE COMPUTING

ACKNOWLEDGEMENT

It is my proud privilege to present a project on **“Electricity Bill Payment”**. I take this opportunity to express deep sense of gratitude and would like to give thanks to my guide, **“Kapil Agrawal”, PBL Guide**, School of Advanced Computing, Sanjeev Agrawal Global Educational University, Bhopal, M.P. for his valuable guidance, inspiration and encouragement that has led to successful completion of this work.

I could not have accomplished, what I actually have, without their guidance. I would like to express my heartfelt thanks to **Dr. Pinaki Ghosh, HOD, School of ADVANCE COMPUTING**, for his valuable suggestions throughout the project work.

I would like to express my heartfelt thanks and sense of gratitude to **“Kapil Agrawal”** for being a constant source of inspiration. I am also thankful to all faculty members and staff of School of Advanced Computing for their suggestions and support.

I would like to deeply thank my family and friends for all the support and encouragement they have rendered time to time.

Last but not the least, I dedicate my work to almighty God without whose wish and helping hands this work would not have taken the shape it has now and also to my family members whose support and encouragement had led me to complete this task.

Piyush Sahu – 230725

Tanishka Soni – 231065

Agniva Jana Bhattacharya - 232528

Jatin Singh -231628

Vivek Singh Chouhan-231528

ABSTRACT

Electricity is an energy that play a major role in human life. In day to day life, each and every device from machinery to wrist watch everything works on electricity. It is the most basic requirement next to food, shelter, and clothing. From the past decade's lot of changes took place in electricity departments but even now they are using manual billing system. This system has a wide range of disadvantages like malpractices are done while billing, escaping from punishment if any late payments, manpower for billing and collecting bills and wastage of paper billing. And moreover, if a fire accident or a technical problem arises the whole lane (transformer) will be terminated from power supply this may cause an inconvenience to the peer consumers too. Here, we are concerned about the economic loss that arises due to the manual billing system. In the manual billing system, every month end or for a couple of months bill is generated. An employee from the electricity department comes to each and every house for billing the meters based on the number of units the consumer has consumed. Hence, In this project, we propose and discuss a new adaptive mechanism which reduces all the above-mentioned losses.

TABLE OF CONTENTS

Title page	i
Certificate	ii
Certificate of Approval	iii
Declaration	iv
Acknowledgement	v
Abstract	vi
Table of Content	vii
List of Figures	viii
List of Tables	ix
CHAPTER	NAME OF CONTENT
CHAPTER-1	INTRODUCTION
1.1	PROJECT AIMS AND OBJECTIVES
1.2	BACKGROUND OF THE PROJECT
CHAPTER-2	SYSTEM STUDY
2.1	SCOPE
2.2	FEATURES
2.3	DATA STRUCTURE
CHAPTER-3	ABOUT CODE
3.1	Programing Language Used
3.2	Terms usesd
CHAPTER-4	PROBLEM IDEFINITION & PROPOSED WORK
4.1	Data storage
4.2	Consumers interaction
4.3	Consumers functions

Chapter-01

Introduction

This chapter gives an overview about the aim and objectives background and operation environment of the system.

1.1PROJECT AIMS AND OBJECTIVES:

The project aims and objectives that will be achieved after completion of this project are discussed in this subchapter. The aims and objectives are as follows:

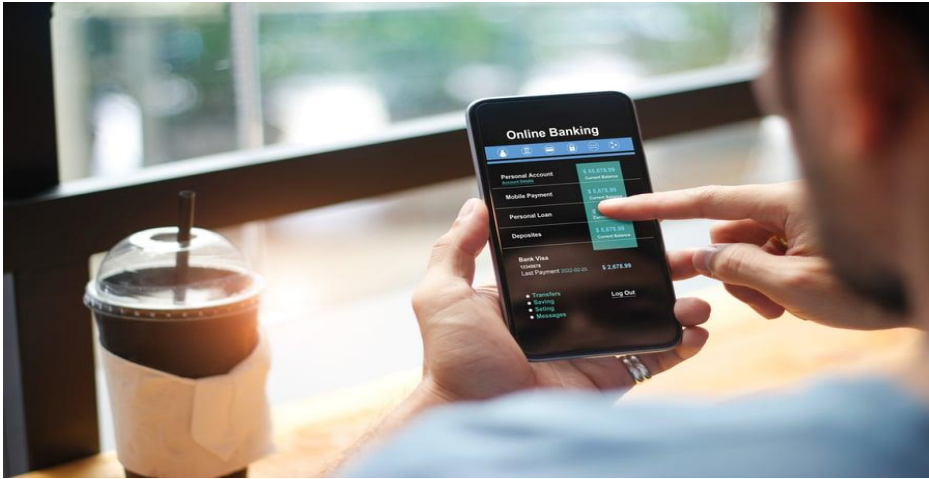
- Online Bill Payment
- Overview of monthly bill
- Overview of all pending bills
- Consumer get a consolidate report of how many bills are pending and paid is a month along with amount.

After Registration: (consumer can perform following operations)

- Edit Details (name, mobile number & email)
- Change Password
- View current bill
- See bill history
- Pay current bill from wallet
- logout

1.2 Background of project :

E-electricity bill payment is a process by which a consumer can pay online. It also helps, protect customer's privacy and identity by eliminating any risk associated with lost or stolen paperwork. Customer will no longer have to pay for cheques, stamps or envelopes. Customer can save on the fuel costs too. By eliminating paper bills and cheques, one can reduce ones impact on the environment as well.



*Paying Bill Is Secure now
ECOLOGICAL bill payment.*

2.1 Scope:

Our project aims at Business process automation, i.e. we have tried to computerize various processes of Electricity Billing System. In the sector of electricity board we have computerizes their department and stock maintenance.

Scope of any software depends upon the following things:

1. It should satisfy the user requirement.
2. Be easy to understand by the user and the operator.
3. Be easy to operate.
4. Have a good user interference.
5. Be expandable.
6. Delivered on schedule within the budget.

We have tried to make such type of software, which satisfy the above given requirement.

2.2 Features:

Authentication:

- Consumers and admins can log in with their respective credentials.
- Consumer registration option available if login fails.

Consumer Functionalities:

- Edit personal details (name, mobile number, email).
- Change password.
- Add money to wallet.
- View current bill for a specific month.
- See bill payment history.
- Pay current bill from the wallet.
- Logout.

Admin Functionalities:

- View all consumers.
- Search for a consumer by mobile number.
- Add a monthly bill for consumers.

- View the bill of a consumer for a specific month.
- View all pending bills for consumers.
- Get a consolidated report of pending and paid bills in a month.
- View transaction history by date range.
- Logout.

2.3 Data Structures:

consumer: Stores consumer information.

admin: Stores admin information.

bill: Stores bill information (consumer name, bill month, amount, status).

wallet: Stores wallet amount for consumers.

pendingBills: Stores pending bills.

2.4 Operations:

Calculating Bill Amount: Based on units consumed.

Calculating Adjustment: Based on previous and current amounts.

Getting Previous Month: Helper function to find the previous month.

Getting Previous Month's Bill: Retrieves the bill for a consumer from the previous month.

Admin Operations: CRUD operations for managing consumers and bills.

Consumer Operations: Manage personal details, view bills, pay bills, add money to wallet.

CHAPTER-03:

ABOUT CODE

3.1 Programing Language Used:

The language we used to create this project is **javascript**. JavaScript was invented by Brendan Eich in 1995.

It was developed for Netscape 2, and became the ECMA-262 standard in 1997.

After Netscape handed JavaScript over to ECMA, the Mozilla foundation continued to develop JavaScript for the Firefox browser. Mozilla's latest version was 1.8.5. (Identical to ES5).

3.2 Terms Used in code:

- **Functions**
- **Switch case**
- **IF else ladder**
- **For loops**
- **Do-while loops**
- **Var , let , const**
- **Objects**
- **Arrays**
- **Array methods**
- **Break , continue statements**

Functions: A JavaScript function is defined with the function keyword, followed by a **name**, followed by parentheses **()**.

Function names can contain letters, digits, underscores, and dollar signs (same rules as variables).

The parentheses may include parameter names separated by commas:
(parameter1, parameter2, ...)

The code to be executed, by the function, is placed inside curly brackets: **{ }**

```
function name(parameter1, parameter2, parameter3) {  
  // code to be executed  
}
```

Function **parameters** are listed inside the parentheses () in the function definition.

Function **arguments** are the **values** received by the function when it is invoked.

Inside the function, the arguments (the parameters) behave as local variables.

The () operator invokes (calls) the function:

Example:

Convert Fahrenheit to Celsius:

```
function toCelsius(fahrenheit) {  
  return (5/9) * (fahrenheit-32);  
}
```

```
let value = toCelsius(77);
```

Switch-case:

The switch statement executes a block of code depending on different cases.

The switch statement is a part of JavaScript's "Conditional" Statements, which are used to perform different actions based on different conditions. Use switch to select one of many blocks of code to be executed. This is the perfect solution for long, nested if/else statements.

The switch statement evaluates an expression. The value of the expression is then compared with the values of each case in the structure. If there is a match, the associated block of code is executed.

The switch statement is often used together with a break or a default keyword (or both). These are both optional:

The **break** keyword breaks out of the switch block. This will stop the execution of more execution of code and/or case testing inside the block. If break is omitted, the next code block in the switch statement is executed.

The **default** keyword specifies some code to run if there is no case match. There can only be one default keyword in a switch. Although this is optional, it is recommended that you use it, as it takes care of unexpected cases.

Syntax:

```
switch(expression) {  
  case x:  
    // code block  
    break;  
  case y:  
    // code block  
    break;  
  default:  
    // code block  
}
```

IF-Else statements:

The if/else statement executes a block of code if a specified condition is true. If the condition is false, another block of code can be executed.

The if/else statement is a part of JavaScript's "Conditional" Statements, which are used to perform different actions based on different conditions.

In JavaScript we have the following conditional statements:

- Use **if** to specify a block of code to be executed, if a specified condition is true
- Use **else** to specify a block of code to be executed, if the same condition is false
- Use **else if** to specify a new condition to test, if the first condition is false

```
if (condition1) {  
  // block of code to be executed if condition1 is true  
} else if (condition2) {  
  // block of code to be executed if the condition1 is false and condition2 is true  
} else {  
  // block of code to be executed if the condition1 is false and condition2 is  
  false  
}  
}
```

loops:

Loops are used in JavaScript to perform repeated tasks based on a condition. Conditions typically return true or false. A loop will continue running until the defined condition returns false.

for Loop:

Syntax

```
for (initialization; condition; finalExpression) {  
  
  // code  
  
}
```

The for loop consists of three optional expressions, followed by a code block:

- **initialization** - This expression runs before the execution of the first loop, and is usually used to create a counter.
- **condition** - This expression is checked each time before the loop runs. If it evaluates to true, the statement or code in the loop is executed. If it evaluates to false, the loop stops. And if this expression is omitted, it automatically evaluates to true.
- **Final Expression** - This expression is executed after each iteration of the loop. This is usually used to increment a counter, but can be used to decrement a counter instead. Any of these three expressions or the code in the code block can be omitted.

for loops are commonly used to run code a set number of times. Also, you can use `break` to exit the loop early, before the condition expression evaluates to false.

do...while loop:

Syntax:

```
do {  
  
    // statement  
  
} while (condition);
```

The do...while loop is closely related to while loop. In a do...while loop, condition is checked at the end of each iteration of the loop, rather than at the beginning before the loop runs. This means that code in a do...while loop is guaranteed to run at least once, even if the condition expression already evaluates to true.

Objects:

An object is a collection of properties, and a property is an association between a name (or key) and a value. A property's value can be a function, in which case the property is known as a method.

```
let person = {firstName:"John", lastName:"Doe", age:50, eyeColor:"blue"};
```

CHAPTER-04:

Working of Code

4.1 Data Storage:

Four main arrays hold essential information:

consumer: Contains consumer details (name, mobile number, email, password).

admin: Stores admin credentials (name, password).

bill: Keeps track of consumer bills (name, month, amount, status).

wallet: Records the amount in the consumer's wallet.

4.2 Consumer's Interaction:

The code uses the readline-sync module for user input.

It offers multiple functions for various actions:

The **ReadLine** method reads a line from the standard input stream. (For the definition of a line, see the paragraph after the following list.) This means that: If the standard input device is the keyboard, the ReadLine method blocks until the user presses the Enter key.

User Registration (registerConsumer()): Allows new consumers to register with their details. It Takes the essential details from the user—name, mobile number, email, and password—via interactive prompts. Upon gathering this information, it structures it into a new object updating the user's data. This newly formed object is then integrated into the

c
o
n
s
u
m
e
r

[

]

```
125 ✓ function registerConsumer() {  
126   // User input for registration details  
127   let name = input.question("Enter your name: ");  
128   let mobileNo = input.questionInt("Enter your mobile number: ");  
129   let email = input.question("Enter your email: ");  
130   let password = input.question("Enter your password: ");  
131  
132   // Adding the new consumer to the consumer array  
133   consumer.push({  
134     consumerName: name,  
135     mobileNo: mobileNo,  
136     consumerEmail: email,  
137     consumerPassword: password,  
138   });  
}
```

Consumer Login (consumerLogin()):

a

r

a

y

,

It Verifies consumer credentials and allows login. The `consumerLogin()` function manages consumer authentication within the electricity bill payment system. It begins by prompting users to input their name and password through interactive prompts. Upon receiving this data, it conducts a comparison between the provided credentials and the existing consumer records stored in the `consumer` array. It iterates through the array, verifying if there's a match for both the entered name and password. If a match is found, indicating successful authentication, it triggers a "Login Successful" message and navigates the user to the `consumerMenu()`, offering various functionalities within the system. In case of failed authentication, it notifies the user about the invalid credentials and prompts them if they wish to register. If the user opts to register, it directs them to the `registerConsumer()` function; otherwise, it redirects them to the main menu via `mainMenu()`. This function serves as the entry point for consumers, allowing access to the system's features upon successful authentication while providing an avenue for registration if needed.

Consumer Menu (consumerMenu()):

It Provides a menu of options for consumers to perform actions like viewing bills, adding money to the wallet, etc.

The `consumerMenu()` function acts as the interactive hub for consumers within the electricity bill payment system, presenting a menu of options via console output. Each option corresponds to a specific action that users can perform. Upon displaying the menu, it prompts users to input their choice, expecting a numeric selection. Based on the chosen action, the function triggers different functionalities. (183 - 223)

```
183  function consumerMenu() {  
184      console.log("1. Edit details");  
185      console.log("2. Change password");  
186      console.log("3. Add money to wallet");  
187      console.log("4. View current bill");  
188      console.log("5. See bill history");  
189      console.log("6. Pay current bill from wallet");  
190      console.log("7. Logout");  
191  
192      // User choice for actions  
193      let choice = input.questionInt("Enter your choice: ");  
194  >  switch (choice) { ... }  
223  }
```

Admin Login (adminLogin()):

The `adminLogin()` function manages the authentication process specifically for administrators within the electricity bill payment system. It initiates by prompting the admin to input their credentials—namely, their name and password—through interactive console prompts. Subsequently, the function verifies these provided credentials against the existing admin records stored within the `admin` array.

```
373 ✓ function adminLogin() {  
374     // Get admin credentials from user input  
375     adminName = input.question("Enter your name: ");  
376     adminPassword = input.question("Enter your password: ");  
377   Q   let flag = false;  
378  
379     // Check if entered credentials match any admin in the system  
380 ✓   for (let i = 0; i < admin.length; i++) {  
381       if (  
382         adminName == admin[i].adminName &&  
383         adminPassword == admin[i].adminPassword  
384 ✓     ) {  
385         flag = true;  
386         break;  
387       }  
388     }  
389  
390     // If credentials are valid, display success message and go to admin menu  
391 ✓   if (flag == true) {  
392     console.log("Login Successful");  
393     adminMenu();  
394 ✓   } else {  
395     // If credentials are invalid, display error message  
396     console.log("Invalid Credentials");  
397   }  
398 }
```

It conducts a comparison by iterating through the array, checking for a match between the entered admin name and password combination with the stored admin credentials. Upon successful authentication, denoted by a match between the provided and stored data, the function displays a "Login Successful" message, granting access to the admin functionalities by navigating to the `adminMenu()`. In the event of authentication failure, when the entered credentials don't align with any stored admin data, the function notifies the user about the invalid login attempt.

This function forms the initial gateway for admin access, ensuring secure entry into the system and providing access to administrative features upon successful verification while alerting in case of failed authentication.(373 - 398);

Admin Menu (adminMenu()):

The `adminMenu()` function is like a control center for admins in the electricity bill system. It shows a menu on the screen with different options, each doing a different job for managing things. When an admin sees this menu, they can type a number to pick what they want to do. For example, they can see a list of all users, find a user by their phone number, add bills for

users each month, and more. Once they pick something by typing a number, the function does that specific job they chose. It's like a remote control with different buttons for an admin to do different tasks in the system.

Menu after successful Admin Login:

```
Login Successful
1. View all consumers
2. Search consumer by mobile number
3. Add monthly bill for consumers
4. View the bill of the consumer for month
5. View all pending bills for consumers
6. Get a consolidate report of how many bill are pending and paid in a month along with amount
7. Logout
8. View transaction history by date range
Enter your choice: █
```

4.3 Consumer Functions:

Edit Details (editDetails()):

The `editDetails()` function is responsible for letting users change their personal information within the electricity bill payment system. It prompts users to input updated details such as their name, mobile number, and email through interactive prompts. Once the new information is provided, the function goes through the `consumer` array to find the user's details and updates them with the newly provided information. Essentially, it helps users modify their stored information, ensuring their details remain accurate and up-to-date within the system. This function ensures users have control over their personal data, allowing them to make necessary changes whenever needed, contributing to a smoother and more personalized experience within the system.

Add Money to Wallet (`addMoneyToWallet()`):

The `addMoneyToWallet()` function helps users put more money into their wallet for paying electricity bills. First, it asks users how much money they want to add. Once users say how much, it looks in the system to find their wallet. Then, it adds the money they said into their wallet. It tells users it worked and sends them back to the main menu. This helps users make sure they have enough money in their wallet to pay their bills and makes it easier for them to use the system.

Change Password (`changePassword()`):

```

275 ✓ function addMoneyToWallet() {
276     // User input for the amount to add
277     let amount = input.questionInt("Enter amount to add in wallet: ");
278
279     // Updating the wallet amount
280 ✓   for (let i = 0; i < wallet.length; i++) {
281 ✓       if (consumerName === wallet[i].consumerName) {
282           wallet[i].walletAmount += amount;
283       }
284   }
285
286   // Success message
287   console.log("Amount added successfully");
288
289   // Redirecting to the consumer menu
290   consumerMenu();
291 }
---
```

The `changePassword()` function enables users to update their login credentials within the electricity bill payment system. It begins by prompting users to input their old password and the new password they wish to use.

Following code change password in this program:

```

249 ✓ function changePassword() {
250     // User input for old and new passwords
251     let oldPassword = input.question("Enter your old password: ");
252     let newPassword = input.question("Enter your new password: ");
253     let confirmPassword = input.question("Confirm your new password: ");
254
255     // Checking and updating the password
256 ✓   if (newPassword === confirmPassword) {
257 ✓       for (let i = 0; i < consumer.length; i++) {
258 ✓           if (consumerName === consumer[i].consumerName) {
259               consumer[i].consumerPassword = newPassword;
260           }
261       }
262
263       // Success message
264       console.log("Password Updated Successfully");
265 ✓   } else {
266       // Password mismatch error
267       console.log("Password does not match");
268   }
269
270   // Redirecting to the consumer menu
271   consumerMenu();
272 }
---
```

View Bills (viewCurrentBill()):

The `viewCurrentBill()` function allows users to see their current electricity bill within the system. It presents users with a prompt asking them to input the month for which they want to view the bill. Once the user enters the month, the function traverses through the `bill` data structure to find the bill associated with the current user (`consumerName`) and the specified month (`billMonth`).

View Bill History (seeBillHistory()):

The `seeBillHistory()` function allows users to view their billing history within the electricity bill payment system. When users choose this option from the consumer menu, the function retrieves and displays a record of all the bills associated with the current user (`consumerName`) from the `bill` data structure.

It iterates through the `bill` array, checking each bill entry to find those that match the current user's name. For each bill that matches the user, it displays the details of that bill, such as the month, bill amount, and status (paid or pending), on the console.

After displaying all the bill details for the user, the function redirects the user back to the consumer menu, ensuring a smooth continuation of their interaction with the system.

code to see bill history:

```
316 // Function to view the bill history for a consumer
317 function seeBillHistory() {
318     // Displaying the bill details for the consumer
319     for (let i = 0; i < bill.length; i++) {
320         if (consumerName === bill[i].consumerName) {
321             console.log(
322                 `Your bill for ${bill[i].billMonth} is ${bill[i].billAmount} and status is ${bill[i].billStatus}`
323             );
324         }
325     }
326
327     // Redirecting to the consumer menu
328     consumerMenu();
329 }
```

Pay Bills (payCurrentBillFromWallet()):

The `payCurrentBillFromWallet()` function helps users pay their pending electricity bills using the money stored in their wallet. Here's how it works:

1. **Asks for Month:** It first asks users which month's bill they want to pay.
2. **Checks and Pays Bill:** Then it checks if there's an unpaid bill for that month. If there is and the user has enough money in their wallet, it takes the bill amount from their wallet and marks the bill as paid.
3. **Shows Result:** After trying to pay the bill, it tells users if the payment worked or if they didn't have enough money.
4. **Goes Back to Menu:** Finally, it sends users back to the main menu, so they can do other things in the system.

This function makes it easier for users to handle their bills, using the money they've stored in their system wallet.

4.4 Admin's Functions:

View All Consumers (viewAllConsumers()):

The `viewAllConsumers()` function helps administrators see the details of everyone who has signed up for the electricity bill payment system.

Here's how it works:

1. When an admin chooses this option, the function looks into the system's records where all user information is stored.
2. It then goes through each user's details, like their name, phone number, email, and more, showing all this information on the screen for the admin to see.
3. This displayed information helps the admin get an overview of all the users registered in the system, making it easy to manage user accounts and their details.
4. Once all the user details are shown, the function goes back to the admin menu. This allows the admin to perform other important tasks or access different parts of the system.

Search Consumer by Mobile Number (`searchConsumerByMobileNumber()`):

The `searchConsumerByMobileNumber()` function allows administrators to find specific consumers by searching for their details using their mobile number within the electricity bill payment system.

1. **Prompt for Mobile Number:** The function prompts the admin to input the mobile number of the consumer they want to find.
2. **Searches for the Consumer:** Once the admin enters the mobile number, the function searches through the system's records, specifically the `consumer` data structure, to find a match with the provided mobile number.
3. **Displays Consumer Details:** If a consumer's details match the entered mobile number, the function displays all the information associated with that consumer, such as their name, contact details, or any other information stored in the system.
4. **Admin's Information Retrieval:** This functionality helps administrators quickly retrieve specific consumer details based on their mobile number, assisting in managing user accounts or addressing any consumer-related concerns or inquiries.
5. **Return to Admin Menu:** After displaying the consumer's details, the function returns to the admin menu, allowing the admin to continue with other administrative tasks within the system.

Add Monthly Bill for Consumers (`addMonthlyBillForConsumers()`):

The `addMonthlyBillForConsumers()` function is a feature within the electricity bill payment system that allows administrators to add monthly bills for consumers.

Here's how it works :

1. **Input Details:** The function prompts the admin to enter specific details required for generating a bill, including the consumer's name, the billing month, and the bill amount.
2. **Bill Generation:** After receiving the necessary details, the function calculates the total bill amount, considering any pending amounts from the previous month.
3. **Adding Bill Details:** It then adds this newly generated bill to the system's records. This includes the consumer's name, the billing month, the calculated bill amount, and marks the status of the bill as "pending" since it's newly added.
4. **Confirmation Message:** Upon successfully adding the bill details, the function displays a confirmation message, confirming that the bill has been added to the system.
5. **Return to Admin Menu:** Finally, the function returns the admin to the admin menu, allowing them to perform other administrative tasks or access different functionalities within the system.

code for adding monthly bill :

```

488
489 ✓ function addMonthlyBillForConsumers() {
490
491     let name = input.question("Enter consumer name: ");
492     let month = input.question("Enter month: ");
493     let amount = input.questionInt("Enter amount: ");
494     let status = "pending";
495     let previousMonth = getPreviousMonth(month);
496     let pendingAmountFromPreviousMonth = getPendingAmountFromPreviousMonth(name, previousMonth);
497     amount += pendingAmountFromPreviousMonth;
498
499
500 ✓     bill.push({
501         consumerName: name,
502         billMonth: month,
503         billAmount: amount,
504         billStatus: status,
505     });

```

View Consumer Bills (viewBillOfTheConsumerForMonth()):

The **viewBillOfTheConsumerForMonth()** function helps administrators look at a particular consumer's bill for a specific month.

Here's how it works:

1. **Admin Input:** It asks the administrator to enter the consumer's name and the month they want to see the bill for.

2. **Searching:** Once the admin provides this information, the function checks the system's records to find the bill that matches the entered details.
3. **Displaying Information:** If a bill is found, it shows the bill amount and its status (whether it's paid or pending) for the specified consumer and month.
4. **Admin's Access to Information:** This feature allows administrators to quickly check a consumer's bill details for a particular month, making it easier to handle any billing-related questions or concerns.
5. **Return to Admin Menu:** After showing the bill details, it returns to the admin menu, enabling the admin to continue managing the system or perform other administrative tasks.

View Pending Bills (viewAllPendingBillsForConsumers()):

The **viewAllPendingBillsForConsumers()** function allows administrators to see all the pending bills within the electricity bill payment system.

Here's how it works:

1. **Search for Pending Bills:** When this function is used, it looks through the system's records to find bills that have a status marked as "pending."
2. **Compilation of Pending Bills:** It gathers and lists all these pending bills together.
3. **Display of Pending Bills:** Once it collects all the pending bills, it displays this compiled list, showing the details of each pending bill, such as the consumer's name, the bill amount, and any other pertinent information associated with those bills.
4. **Administrator's Oversight:** This function provides administrators with an overview of all pending bills, helping them in managing and addressing pending payments or discrepancies efficiently.
5. **Return to Admin Menu:** After displaying the pending bills, the function allows administrators to return to the admin menu for further management or to perform other administrative actions.

View Transaction History (viewTransactionHistoryByDateRange()):

The **viewTransactionHistoryByDateRange()** function assists administrators in reviewing transaction history within a specified date range in the electricity bill payment system.

Here's how it works:

1. The function prompts administrators to input a start date and an end date to define the desired range for transaction history.
2. Once the date range is provided, the function filters through the system's records to identify transactions that occurred within the specified date range.

3. It displays the details of these transactions, including the date, the consumer involved, the transaction amount, and its status (whether paid or pending).
4. This feature empowers administrators to monitor and analyze transactional activities within the defined timeframe, aiding in auditing or resolving any discrepancies.
5. After presenting the transaction history, the function allows administrators to return to the admin menu, enabling them to continue managing the system or performing additional administrative tasks.

Generate Consolidated Reports (getConsolidateReport()):

Provides a report of pending and paid bills for a specified month.

code to get report of pending bills for specified months:

```

575 // Function to get a consolidated report of pending and paid bills in a month
576 ✓ function getConsolidateReport() {
577     // Get the month for the report from user input
578     ? let month = input.question("Enter month: ");
579     let pendingAmount = 0;
580     let paidAmount = 0;
581
582     // Iterate through bills to calculate pending and paid amounts
583     ✓ for (let i = 0; i < bill.length; i++) {
584     ✓     if (month === bill[i].billMonth) {
585     ✓         if (bill[i].billStatus === "pending") {
586             pendingAmount += bill[i].billAmount;
587     ✓         } else {
588             paidAmount += bill[i].billAmount;
589         }
590     }
591 }

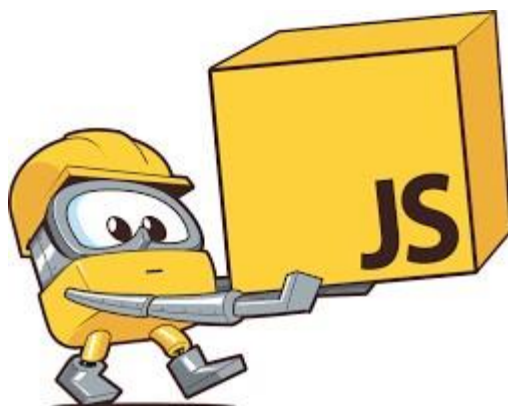
```

4.5 Supporting Functions:

- **calculateBillAmount():** The `calculateBillAmount()` function determines the total cost a consumer owes for their electricity usage. It calculates this by considering fixed charges for service and a variable rate per unit of electricity consumed. Once it computes the base cost, it includes taxes by applying a predetermined tax rate to the total bill amount. This function efficiently generates the overall bill amount, encompassing both fixed and variable charges along with taxes, providing consumers with a comprehensive total for their electricity consumption.



- **calculateAdjustment():** The `calculateAdjustment()` function computes the adjustment or change in the billing amount between two consecutive billing cycles. It takes in the previous amount and the current amount to determine the difference or adjustment. By subtracting the previous amount from the current one, it identifies whether the billing amount has increased or decreased. This function helps in understanding and managing changes in the billed amount over time, aiding in tracking billing adjustments or modifications between different periods.



Overall Flow:

The program starts with the `mainMenu()` function, where users choose between admin and consumer login.

Depending on the user type, it navigates to the respective login functions and subsequent menus for various actions.

- **Initialization:** It begins by initializing data structures for consumers, admins, bills, and wallets, storing relevant information.
- **User Login and Registration:** Users are prompted to log in or register. Upon successful login, consumers access functionalities like viewing bills, adding money to their wallet, etc. If login fails, they can register as a new user.
- **Admin Functionalities:** Admins have access to functions like viewing consumers, adding bills, checking pending bills, and generating reports.
- **Consumer Actions:** Consumers can edit details, change passwords, view and pay bills, add money to their wallet, among other actions.
- **Bill Handling:** The system manages bill-related functionalities such as calculating bills based on consumption, handling bill payments from wallets, and tracking pending bills.
- **Flow Control:** The program navigates users through various menus, directing them to different functionalities based on their choices.
- **Data Management:** Throughout, the system interacts with and manipulates data structures to store and retrieve user information, bill details, and transaction histories.

-----End of report-----